PRD: LGTM

Looks Good To Monitor — WASM 코드 실행 playground + observability stack

기준: 42 subject v1.1 · 작성일: 2026-10-21 · v1.3 · 작성자: 【login】 · 상태: 설계 확정, 개발 착수

1. 개요
2. 확정된 기술 결정
3. 범위
4. 사용자 시나리오
5. 시스템 아키텍처
6. 기능 요구사항
7. 보안 요구사항
8. Observability 요구사항
9. 인프라 / 배포
10. VM 사양
11. README 요구사항
12. 리스크
13. 마일스톤
14. 미결정 사항
15. 피처 백로그

1. 개요

1.1 한 줄 정의

사용자가 브라우저에서 Rust 코드를 작성하고 실행하면, 백엔드가 이를 WASM으로 compile해 WASI sandbox에서 실행하고, 코드와 output을 IPFS에 올려 공유 링크를 돌려주는 playground. 이 전체 흐름을 metrics, traces, logs로 관측한다.

1.2 목표

- **기능**: 안전한 원격 코드 실행과 IPFS 공유
- **학습**: observability 3요소(logs, metrics, traces)와 그 사이의 correlation을 직접 구현하고 설명할 수 있는 수준
- **평가**: 새로 만든 VM에서 `make` 한 번으로 전체가 올라감

1.3 성공 기준

- 새 VM에서 `make` → `lgtm.local`에서 코드 실행 → IPFS 링크 동작 → Grafana에서 metric → exemplar → trace → logs까지 클릭만으로 이동
- 같은 CID가 `ipfs.io/ipfs/<cid>`에서도 열림 (네트워크 조건 충족 시)
- sandbox 테스트 케이스(7.2)와 compile 단계 공격(7.3)이 모두 막힘

2. 확정된 기술 결정

항목	결정	이유 / 비고
로컬 K8s	k3d	가볍고 Traefik ingress 기본 포함. IPFS swarm 포트 4001은 클러스터 생성 시점에 `-p`로 매핑해야 함 (나중에 추가 불가)
실행 대상 언어	Rust	target `wasm32-wasip1`
Backend	Go	OTel Go SDK 성숙
WASI runtime	Wasmtime (`wasmtime-go`)	memory limiter, epoch interruption 제공. cgo 필요 → 빌드 이미지에 gcc 포함

Frontend	React + Monaco	
Traces	Tempo	TraceQL, trace-to-logs 연동
Metrics	Mimir (monolithic)	`-target=all`, filesystem storage, pod 1개. distributed chart는 8GB VM에서 과함
Logs	Loki	
Telemetry 수집	OpenTelemetry Collector (contrib)	receivers → processors → exporters 구조가 OTel 개념 설명에 그대로 쓰임
IPFS	Kubo	
실패 케이스 IPFS 업로드	업로드함	`status.json`을 같이 넣어 링크만 봐도 결과 확인 가능
Compile 방식	`rustc` 단일 파일 + 별도 compiler pod	cargo의 `build.rs`, proc-macro 차단. pod 격리는 7.3 참고
로그 수집 경로	OTLP (+ stdout JSON 병행)	trace_id 자동 부착. 8.4 참고
Trace dashboard 데이터	A안 (앱 histogram + exemplar), Slowest/Longest만 Tempo	8.6 참고

3. 범위

3.1 In scope (mandatory)

영역	내용
Frontend	코드 에디터, 예제 코드, Run 버튼, 로딩 표시, 에러/output 표시, IPFS 링크
Backend	코드 수신 API, WASM compile, WASI 실행, output 캡처, IPFS 업로드, 결과 반환
Sandbox	network/filesystem 차단, memory·time·output 크기 제한
IPFS	자체 노드, `ipfs.lgtm.local` gateway, public gateway에서도 같은 CID로 접근
Observability	Grafana, Loki, Tempo, Mimir, OTel Collector
Dashboard	Metrics 1개, Traces 1개
Infra	VM 안의 k3d 클러스터, Makefile, README

3.2 Out of scope

- 클라우드 서비스 일체 (subject에서 금지)
- 사용자 계정, 인증, 영구 DB
- 프로덕션 수준 HA(고가용성), 멀티 노드. VM 한 대라 의미가 없고 RAM만 더 씀. pod 1개 + K8s 자동 재시작으로 충분

3.3 Bonus (mandatory가 완벽할 때만 평가)

Deep IPFS Integration, Infrastructure Observability, Polyglot, 짧은 URL 공유, Performance Profiling. mandatory 완료 후 재검토.

4. 사용자 시나리오

S1. 정상 실행

1. `lgtm.local` 접속, 예제 코드가 이미 들어있음
2. Run 클릭 → spinner 표시
3. output 표시 + IPFS 링크 노출
4. 링크 클릭 → `ipfs.lgtm.local/ipfs/<CID>/` 에서 `main.rs`, `output.txt`, `status.json` 확인

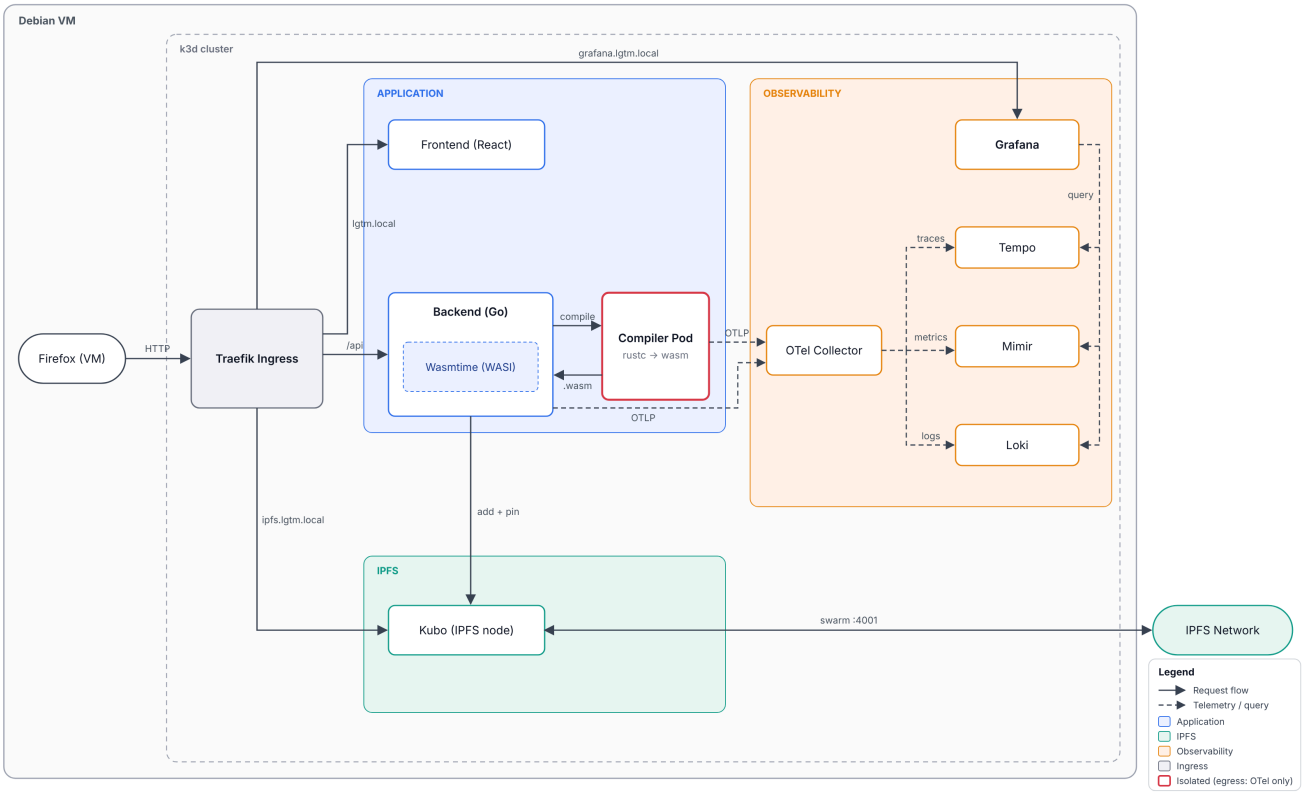
S2. Compile 에러 — rustc 메시지를 그대로, 읽기 좋게 표시. 코드와 에러 메시지를 IPFS에 업로드하고 링크 제공.

S3. Runtime 에러 / 제한 초과

- 무한 루프 → 5초에서 중단, "Time limit exceeded (5s)" 표시
- 과도한 할당 → memory limit 에러
- 출력 폭주 → 10kB에서 잘림 + truncated 표시

S4. 운영자 디버깅 — Metrics dashboard에서 실패 수 급증 확인 → duration 그래프의 튀는 점 클릭 → exemplar로 trace 열기 → 실패한 span 확인 → "Logs for this span"으로 해당 로그 확인 → span attribute의 CID/hash로 실행된 코드 식별.

5. 시스템 아키텍처



- **실선**: 요청 흐름 / **점선**: telemetry(OTLP)와 Grafana 조회
- 디펜스 시연은 **VM 안 브라우저(Firefox)** 기준. `make hosts`는 VM의 `/etc/hosts`만 고치므로, 호스트 PC 브라우저 접속(호스트 hosts 수정 + 포트포워딩)은 README에 옵션으로만 안내
- IPFS Network만 VM 경계 **밖**. 4001 포트가 VM과 공유기를 지나야 외부 gateway에서 CID가 열린다 (NAT 한계 설명 지점)
- Wasmtime은 별도 pod가 아니라 **Backend 프로세스 안**에서 실행. compile만 Compiler Pod(빨간 테두리, 격리)로 분리
- `ipfs.lgtm.local`은 Ingress에서 Kubo gateway로 바로 연결 (Backend를 거치지 않음)
- Collector → Tempo/Mimir/Loki는 수집, Grafana → Tempo/Mimir/Loki는 조회. Grafana가 데이터를 가져가는 방향

> backend → compiler 사이 HTTP 호출에 W3C `traceparent` 헤더가 실려, 두 서비스의 span이 하나의 trace로 묶인다. 디펜스에서 "분산 trace"를 보여줄 수 있는 지점.

6. 기능 요구사항

6.1 Frontend

ID	요구사항	수용 기준
FE-01	Monaco 코드 에디터	Rust syntax highlight, 줄 번호. Monaco는 로컬 번들 (CDN 로드 금지). 브라우저 네트워크 탭에 외부 도메인 요청 0개
FE-02	예제 코드 사전 로드	첫 로드 시 Rust 특징(struct, trait, iterator 등)을 보여주는 예제
FE-03	눈에 띄는 Run 버튼	실행 중 비활성화 (중복 요청 방지)
FE-04	실행 중 시각 피드백	spinner 또는 progress
FE-05	에러 표시	compile 에러 / runtime 에러 / limit 초과를 구분해 표시
FE-06	output 표시	monospace, stdout/stderr 구분, truncated 표시

FE-07	IPFS 링크 2개	로컬 `ipfs.lgtm.local/ipfs/<CID>` + 공개 `https://ipfs.io/ipfs/<CID>`, 새 탭. 로컬 링크는 VM 밖에서 안 열리므로 "share with the world"는 공개 링크로 충족
FE-08	공개 안내 문구	"실행한 코드는 IPFS에 영구 공개됩니다"
FE-09	(권장) trace ID 노출	디버깅·디펜스 시연용
FE-10	429/413 처리	"서버가 바쁩니다", "코드가 너무 깁니다(64KB)" 안내

6.2 Backend API

`POST /api/execute`

```
// Request
{ "language": "rust", "code": "fn main() { ... }" }

// Response 200 (성공)
{
  "status": "success",
  "stdout": "Hello, world!\n",
  "stderr": "",
  "truncated": false,
  "cid": "bafy...",
  "ipfs_url": "http://ipfs.lgtm.local/ipfs/bafy...",
  "trace_id": "4bf92f...",
  "timings_ms": { "compile": 23, "execute": 5, "ipfs_upload": 142 }
}

// Response 200 (실행 실패)
{
  "status": "compile_error | runtime_error | timeout | memory_limit",
  "message": "...",
  "stderr": "...",
  "cid": "bafy...",          // 실패도 업로드
  "ipfs_url": "...",
  "trace_id": "..."
}

// 413 Payload Too Large / 429 Too Many Requests / 400 invalid_request
```

ID	요구사항
BE-01	API로 코드 수신. body 최대 64KB, 초과 시 413
BE-02	compiler 서비스에 코드 전달, `.wasm` 수신
BE-03	Wasmtime으로 WASI 실행 (7.1 제한 적용)
BE-04	stdout/stderr 캡처 (합계 10kB 제한)
BE-05	성공·실패 모두 IPFS 디렉토리로 업로드
BE-06	결과 반환
BE-07	모든 단계 OTel 계측 (8장)
BE-08	`GET /healthz` (K8s probe)
BE-09	동시 실행 제한: 처리 2개 + 대기열 8개, 초과 시 즉시 429

BE-10	IPFS 업로드 10초 timeout. 초과 시 결과는 반환, `ipfs_url: null`, span ERROR

6.3 IPFS

ID	요구사항
IP-01	클러스터 안에 Kubo 노드 1개 (PVC로 데이터 유지)
IP-02	업로드 구조: `/main.rs`, `/output.txt`, `/status.json`을 하나의 디렉토리로 wrap
IP-03	업로드 후 pin
IP-04	gateway를 `ipfs.lgtm.local`로 노출
IP-05	swarm 포트 4001 tcp/udp를 k3d 생성 시 host로 매핑 + VM 포트포워딩
IP-06	`ipfs.io/ipfs/<CID>`에서 같은 CID로 접근
IP-07	NAT 환경일 때 한계를 README와 디펜스에서 설명
IP-08	Kubo API(5001)는 클러스터 내부 Service로만 노출. Ingress는 gateway(8080)만, gateway는 read-only (`Gateway.Writable` 끔). `ipfs.lgtm.local/api/v0/*`는 실패해야 함

7. 보안 요구사항

7.1 실행 sandbox 및 제한값

ID	항목	값	구현 포인트
SEC-01	Network	전면 차단	WASI socket capability 미부여
SEC-02	Filesystem	전면 차단	preopened dir 없음, env/args 상속 안 함
SEC-03	실행 memory	10MB	Wasmtime store limiter (wasm page 64KiB 단위)
SEC-04	실행 time	5초	epoch interruption, wall-clock 기준
SEC-05	Output	10kB	캡처 버퍼 제한, 초과분 버림 + truncated. 쓰기 단계에서 양 자체를 제한해야 함 (wasmtime-go가 stdout을 파일 경로로만 받으면 5초간 GB 단위 디스크 쓰기 가능). 방식은 14장, F-M2-03 전 결정
SEC-06	요청 body	64KB	손으로 쓰는 Rust 단일 파일은 충분
SEC-07	동시 compile	2	4코어 기준. rustc 1개가 코어 1~2개 사용. pod 배치는 SEC-10과 함께 결정
SEC-08	대기열	8	초과 시 429. 무한 대기 시 전부 timeout 남
SEC-09	Compile time	15초	일반 코드 1~3초. 매크로/제네릭 폭발 방어
SEC-10	Compile memory	rustc 1개당 약 1GB	(a) pod 1개 limit 2GB, 동시 2 / (b) replica 2 × 1GB, pod당 동시 1. pod 1개 1GB에 rustc 2개면 각각 약 500MB이고, OOM 시 진행 중이던 compile이 둘 다 실패할 수 있음 (cgroup v2 동작 확인 필요). F-M2-01 전 결정 (14장)
SEC-11	IPFS 업로드	10초	초과 시 링크 없이 결과 반환

compile 옵션: `rustc --target wasm32-wasip1 -C opt-level=1`. opt-level=3은 compile이 느려지고 playground에서 이득이 거의 없음.

원칙: **runtime은 명시적으로 필요한 capability 외에는 아무것도 주지 않는다.** 디펜스에서 어떤 API 호출로 막았는지 코드로 보여줄 수 있어야 한다.

7.2 Sandbox 테스트 케이스 (수용 기준)

- 무한 루프 → 5초 내 종료, `timeout`
- `vec![0u8; 100_000_000]` → `memory_limit`
- `std::fs::read("/etc/passwd")` → 실패
- `TcpStream::connect(...)` → 실패
- 1MB 출력 → 10kB에서 잘림, `truncated: true`
- 동시 요청 20개 → 10개 처리/대기, 나머지 429

7.3 Compile 단계 격리

WASI sandbox는 **실행** 단계만 막는다. `rustc` 단일 파일 compile로 `build.rs`와 proc-macro는 막히지만, 아래는 compile 시점에 sandbox 바깥에서 그대로 실행된다.

```
const A: &str = include_str!("/etc/passwd");   // 호스트 파일 읽기
const B: &str = env!("SOME_SECRET");           // compile 시점 환경변수 읽기
#[path = "/proc/self/environ"] mod x;          // 경로 지정 모듈 로드
```

> 이 프로젝트는 결과를 IPFS에 올리기 때문에, 읽어낸 내용이 전 세계에 영구 공개된다. 그래서 compile을 별도 pod로 분리한다.

compiler pod 설정

설정	막는 것
컨테이너에 rustc + wasm target만	읽을 만한 파일 자체가 없음
`automountServiceAccountToken: false`	K8s API 토큰 노출
`env -i rustc ...`	`env!()`로 환경변수 탈취
`readOnlyRootFilesystem: true` + `emptyDir` 작업 공간	컨테이너 변조
`runAsNonRoot: true`, capabilities drop ALL	권한 상승
NetworkPolicy: ingress는 backend만, egress는 OTel Collector OTLP 포트 + DNS(53)만 허용	외부 유출. Collector 허용은 rustc span 전송(TR-05)용
resources limit: CPU 2, memory는 SEC-10 결정에 따름	자원 고갈

요청마다 임시 디렉토리를 만들고 끝나면 삭제해, 이전 사용자의 코드가 남지 않게 한다.

수용 기준 ("막힘"의 정의) — M2 전에 택1 (14장)

안	정의	비고
(a)	compile은 성공해도 됨. 단 VM 호스트 파일, backend 환경변수, SA token이 결과에 나오지 않음	격리만으로 충족. 컨테이너 기본 `/etc/passwd`는 노출됨을 디펜스에서 설명
(b)	(a) + 소스에서 `include_str!`, `include_bytes!`, `env!`, `option_env!`, `#[path]`를 deny-list로 거부 (`compile_error` 반환)	심층 방어. 문자열 매칭이라 우회 가능성은 설명 필요

8. Observability 요구사항

8.1 컴포넌트

Grafana, Loki, Tempo, Mimir(monolithic), OTel Collector(contrib). 애플리케이션(backend, compiler)은 OTLP로 Collector에만 보내고, Collector가 각 backend로 분배한다.

8.2 Metrics (OTel Metrics API)

이름	타입	label	용도
`lgtm_executions_total`	Counter	`status`, `language`	성공/실패/전체
`lgtm_execution_duration_seconds`	Histogram	`status`, `language`	duration 그래프 + exemplar. 값 = 요청 전체(root span) duration
`lgtm_stage_duration_seconds`	Histogram	`stage`	compile/execute/ipfs_upload 분포
`lgtm_output_bytes`	Histogram		권장
`lgtm_queue_rejected_total`	Counter		429 발생 수

histogram bucket은 5초 limit 근처를 촘촘하게: `0.05, 0.1, 0.25, 0.5, 1, 2, 3, 4, 4.5, 5, 5.5, 10, 20`. CID, 코드 hash 같은 high-cardinality 값은 label에 넣지 않는다 (trace attribute로).

- **Export 주기**: Go SDK PeriodicReader 기본값은 60초라 대시보드와 exemplar가 최대 1분 늦게 뜬다. reader interval 5초, dashboard refresh 5초로 설정
- **Exemplar 활성화**: otel-go 버전에 따라 `OTEL_GO_X_EXEMPLAR=true`가 필요할 수 있음. 확인 필요
- **저장 이름**: OTLP → Mimir 변환 시 counter에 `_total`, 단위에 `_seconds` 같은 접미사가 자동으로 붙을 수 있다. 위 표와 8.6 쿼리 이름은 F-M1-04에서 Mimir에 실제 저장된 이름을 조회해 맞춘다

8.3 Traces (OTel Tracing API)

```
POST /api/execute              (root, backend HTTP server span)
├── compile                    (backend client span)
│   └── rustc                  (compiler pod server span, traceparent로 연결)
├── execute                    (Wasmtime 실행)
└── ipfs.upload                (Kubo add + pin)
```

ID	요구사항
TR-01	root, compile, execute, ipfs.upload span 필수

TR-02	실패 시 해당 span과 root span status = ERROR, 에러 메시지를 event로 기록
TR-03	코드 식별: `code.sha256`, `ipfs.cid`, `code.language`, `code.size_bytes`
TR-04	실행 결과: `exec.exit_code`, `exec.timed_out`, `exec.output_truncated`, `exec.memory_limited`
TR-05	backend → compiler 호출에 W3C traceparent 전파
TR-06	root span 이름을 코드에서 `POST /api/execute`로 명시 (otelhttp 기본값에 의존하지 않음). 8.6 TraceQL 쿼리가 이 이름에 의존

8.4 Logs

Go `slog`에 handler 2개를 연결한다.

1. **OTel bridge (`otelslog`)** → OTLP → Collector → Loki. trace_id, span_id 자동 부착. "Logs for this span"은 이 경로로 해결
2. **JSON stdout handler** → `kubectl logs`로 즉시 확인, 디버깅용

	OTLP	pod 로그 수집
흐름	앱 → SDK → Collector → Loki	앱 → stdout → `/var/log/pods` → Collector(DaemonSet) → Loki
trace_id	자동	앱이 JSON에 넣고 Collector가 파싱
panic 시	마지막 로그 유실 가능	남음
다른 pod 로그	못 가져옴	전부 가져옴

pod 로그 수집(DaemonSet)은 mandatory에서 생략, Bonus Infrastructure Observability 때 추가.

8.5 Correlation

연결	필요한 설정 (전부 Grafana provisioning 파일로 코드화)
Metric → Trace (exemplar)	앱 histogram에 exemplar 기록 → Collector가 OTLP/remote write로 Mimir에 전달 (exemplar 포함) → Mimir `max_global_exemplars_per_user` 설정 → Grafana Mimir datasource의 exemplar → Tempo 링크
Trace → Logs	OTLP로 들어간 trace_id / span_id는 로그 본문이 아니라 Loki **structured metadata**에 저장된다. 기본 필터는 본문 문자열 검색(`|="<traceId>"`)이라 0건이 나옴. Tempo datasource `tracesToLogsV2`에서 `customQuery: true`, 쿼리에 `| trace_id="${__trace.traceId}" \| span_id="${__span.spanId}"` 사용. Grafana 버전별 차이는 F-M1-03에서 실측
Logs → Trace	Loki datasource의 derived field: trace_id → Tempo. 본문 정규식 매칭이 아니라 `matcherType: label`로 structured metadata의 trace_id를 사용
Trace → Metrics	(권장) Tempo datasource의 tracesToMetrics

> 수동 클릭으로 설정하면 새 VM에서 사라진다. datasource, dashboard 모두 provisioning으로 `make` 한 번에 적용.

8.6 Dashboard

Dashboard 1: Metrics

패널	쿼리 (Mimir)
성공 실행 수	`sum(lgtm_executions_total{status="success"})`
실패 실행 수	`sum(lgtm_executions_total{status!="success"})`
전체 실행 수	`sum(increase(lgtm_executions_total[$__range]))`
(권장) 단계별 p95, 상태별 비율, 429 수	

raw counter를 그대로 쓰면 backend가 재시작될 때 0으로 떨어진다. stat 패널은 `increase(...[$__range])`, 시계열은 `increase` 또는 `rate` 기준. 성공/실패 패널도 같은 방식.

Dashboard 2: Traces — A안 기본, 정확한 값이 필요한 패널만 Tempo

패널	데이터 소스	쿼리
Trace duration over time	Mimir	`histogram_quantile(0.95, sum by (le) (rate(lgtm_execution_duration_seconds_bucket[1m])))` + exemplar ON
Slowest executions	Tempo	TraceQL 테이블: `{ name = "POST /api/execute" && duration > 1s }`. Tempo search는 duration 순이 아니라 최근 순, 기본 20개만 반환하므로 limit을 늘리고 Grafana table transformation으로 duration 내림차순 정렬
Mean duration	Mimir	`rate(..._sum[5m]) / rate(..._count[5m])`
Longest duration	Tempo	TraceQL metrics `max_over_time(duration)`

- A안 이유: exemplar가 subject 핵심 요구사항. 앱 histogram에 붙이는 게 가장 직접적이고 고장 지점이 적음
- 디펜스 설명: "Trace duration over time"을 trace가 아니라 metric으로 그리는 이유 — histogram 값이 root span duration과 같고(8.2), 모든 요청을 집계하므로 샘플링과 무관하며, exemplar로 개별 trace에 바로 연결된다
- Longest를 Tempo로 하는 이유: histogram은 bucket에 세기만 해서 정확한 최댓값을 모름
- TraceQL metrics는 Tempo `local-blocks` processor 필요. `max_over_time` 지원 여부는 배포 버전 문서에서 확인. 안 되면 촘촘한 bucket으로 대체
- dashboard JSON은 `dashboards/`에 두고 provisioning으로 자동 로드
- subject는 duration graph를 복수로 요구. 모든 duration 시계열 패널에서 exemplar 클릭 → trace가 열려야 함. Mean 패널은 _sum/_count라 exemplar가 안 붙을 수 있으므로 bucket 쿼리를 exemplar용으로 겹쳐 그림 (확인 필요, F-M5-03 완료 기준)

9. 인프라 / 배포

ID	요구사항
INF-01	모든 것은 VM 안에서 실행
INF-02	VM 안 k3d 클러스터
INF-03	`Makefile` 이 패키지 설치 + 전체 배포 수행. docker 그룹 추가 후 재로그인 없이 같은 `make` 안에서 다음 단계가 동작해야 함 (`sg docker` 등)
INF-04	호스트명: `lgtm.local` , `grafana.lgtm.local` , `ipfs.lgtm.local` (Traefik Ingress + VM의 `/etc/hosts`). 시연은 VM 안 Firefox, 호스트 PC 접속은 README 옵션
INF-05	새 VM에서 동작 보장 (amd64, arm64 자동 감지)
INF-06	클라우드 서비스 사용 금지
INF-07	이미지는 로컬 빌드 후 `k3d image import` (외부 registry push 안 함)
INF-08	모든 pod에 resources request/limit 지정

9.1 Makefile 타겟

```
make            # = install + cluster + build + deploy + hosts
make install    # docker, kubectl, helm, k3d 설치 (idempotent, arch 감지)
make cluster    # k3d 생성: 80/443 + 4001 tcp/udp 포트 매핑 포함
make build      # backend / compiler / frontend 이미지 빌드 + k3d image import
make deploy     # observability → ipfs → app 순서
make hosts      # /etc/hosts 등록
make status     # pod / ingress 상태
make verify     # scripts/verify/* 전체 실행
make ipfs-check # public gateway 조회 + NAT 진단
make clean / fclean / re
```

9.2 디렉토리 구조

```
.gitignore
Makefile
README.md
backend/        # Go API (+ compiler 서비스)
frontend/       # React + Monaco
k8s/            # manifests, Helm values, NetworkPolicy
dashboards/     # Grafana dashboard JSON + provisioning
docs/           # subject.pdf, PRD.pdf (Claude Code 참조용)
scripts/        # verify/ 피처별 검증 스크립트
CLAUDE.md       # Claude Code 규칙 (짧게 유지)
HANDOFF.md      # 피처 세션 인계 기록
```

subject의 디렉토리 예시에 없는 항목이지만 예시일 뿐이라 감점 요인은 아니라고 판단. CLAUDE.md와 HANDOFF.md는 README의 AI 사용 내역 근거가 된다.

10. VM 사양

항목	권장	최소	이유

CPU	4코어	2코어	rustc·Go 빌드와 pod 약 10개를 같이 돌림. 2코어면 동시 compile을 1로 낮춰야 함
RAM	**8GB**	**8GB**	pod request 약 2.5GB + compile 순간 최대 2GB(SEC-10 어느 안이든) + 데스크톱·Firefox 약 1GB. 6GB는 빌드와 실행이 겹치면 OOM 위험
디스크	**40GB** (동적 할당)	30GB	실사용 약 15GB + OS. k3d는 이미지를 Docker와 노드 containerd에 두 번 저장하므로 실측 확인 필요

ISO

- Intel PC (클러스터 iMac, 리눅스 PC): Debian 13.x `amd64-netinst`
- Apple Silicon Mac: VirtualBox 대신 UTM + Debian 13.x `arm64-netinst`. Makefile이 arm64 자동 감지
- 받는 곳: `https://cdimage.debian.org/debian-cd/current/`

> UTM 기본 네트워크(Shared)는 NAT라 IPFS 4001을 외부에서 받으려면 UTM 포트포워딩 + 공유기 포트포워딩 두 겹이 필요. README IPFS 한계 설명에 따로 적는다.

11. README 요구사항

- 첫 줄 이탤릭: *This project has been created as part of the 42 curriculum by 【login】.*
- **Description**: 목표, 개요
- **Instructions**: VM 사양, 설치, `make`, hosts 설정, 접속 주소
- **Resources**: 참고 문서 + AI를 어떤 작업, 어떤 부분에 썼는지. Claude Code의 `Co-Authored-By` commit trailer를 끄지 않는다. subject가 요구하는 "AI 생성 부분 명시"의 근거가 됨
- 호스트 PC 브라우저로 접속하는 방법 (호스트 hosts 수정 + 포트포워딩)은 옵션으로 안내
- 추가: 아키텍처 다이어그램, 기술 선택 이유(2장), 보안 모델(7장), IPFS NAT 한계

12. 리스크

리스크	영향	대응
NAT 뒤라 public gateway에서 CID 안 열림	IP-06 실패	4001 포트포워딩, IPv6, 디펜스 때 모바일 핫스팟. 안 되면 원인 설명 준비
compile 단계 호스트 정보 유출	보안 구멍 + IPFS 영구 공개	7.3 compiler pod 격리
exemplar가 Grafana에 안 뜸	평가 핵심 실패	M1에서 hello 앱으로 exemplar 파이프라인부터 검증
Mimir 리소스 과다	OOM	monolithic 단일 pod, filesystem storage
Rust toolchain 이미지 크기 / compile 지연	빌드 시간, UX	slim 베이스 + 필요한 target만, opt-level=1
wasmtime-go cgo 빌드 실패	backend 빌드 불가	multi-stage Dockerfile, 빌드 stage에 gcc, arm64도 확인
새 VM에서 Makefile 실패	즉시 감점	깨끗한 VM 스냅샷으로 매번 재검증

13. 마일스톤

단계	내용	완료 기준
M0	VM + k3d + Ingress + hosts, Makefile 뼈대	세 호스트명으로 더미 페이지 접속
M1	관측성 스택 + hello 앱으로 correlation 검증	metric → exemplar → trace → logs 클릭 이동 성공
M2	compiler pod + Wasmtime 실행	7.2, 7.3 테스트 전부 통과
M3	Kubo + 업로드 + 외부 gateway	`ipfs.io`에서 CID 열림
M4	Frontend	S1~S3 시나리오 동작
M5	계측 마무리 + dashboard 2개 provisioning	S4 시나리오 동작
M6	README, 새 VM 재현 테스트	새 VM에서 `make` 한 번으로 전체 동작
M7	(선택) Bonus	

시간이 가장 많이 새는 곳은 코드 실행이 아니라 8.5 correlation 설정과 IPFS 외부 접근. 그래서 M1을 앞에 둔다.

14. 미결정 사항

항목	결정 시점	메모
배포 방식: Helm chart vs raw manifest	F-M1-01 kickoff 전	Mimir monolithic은 공식 chart가 distributed 위주라 raw manifest가 필요할 수 있음. 확인 필요
Collector → Mimir 경로: OTLP endpoint vs remote write	F-M1-01 kickoff 전	버전별로 exemplar 전달 여부 확인 필요
Collector → Loki 경로	F-M1-01 kickoff 전	contrib의 loki exporter는 deprecated로 알려짐. Loki native OTLP가 유력, 확인 필요
Grafana 로그인: anonymous viewer vs admin secret	F-M1-02 전	
compiler pod 메모리 배치 (SEC-10 (a)/(b))	F-M2-01 kickoff 전	(a) pod 1개 2GB, 동시 2 / (b) replica 2 × 1GB, pod당 동시 1. 10장 RAM 계산은 둘 다 최대 2GB
7.3 수용 기준 (a) / (b)	F-M2-02 전	7.3 표 참고
Output 제한 방식 (SEC-05)	F-M2-03 전	wasmtime-go 최신 버전의 writer API 유무 확인. 없으면 `emptyDir{medium: Memory, sizeLimit}` 같은 크기 제한 공간에 쓰기, 또는 쓰는 양 자체를 제한하는 다른 방식

15. 피처 백로그

피처 하나 = Claude Code 세션 하나 = 브랜치 하나 (`feat/m1-02-<이름>`) = draft PR 하나. 각 피처는 plan mode → 실패하는 검증 먼저 커밋 → 구현 → 검증표 → HANDOFF.md 갱신 순서로 진행. 모든 커밋에 Claude Code의 `Co-Authored-By` trailer를 유지한다.

ID	이름	범위	완료 기준	의존
F-M0-01	skeleton-install	디렉토리, .gitignore, Makefile 타겟 뼈대, make install (docker, kubectl, helm, k3d, 버전 고정, arch 감지, idempotent)	scripts/verify/m0-01.sh 통과. 두 번째 make install은 no-op. 재로그인 없이 docker 사용 가능	-
F-M0-02	cluster-ingress-hosts	make cluster (80/443 + 4001 tcp/udp), 세 호스트 더미 페이지, make hosts / status / clean	curl -sf 세 호스트 모두 200. make clean && make cluster 재실행 성공	M0-01
F-M1-01	obs-backends	Tempo, Mimir (monolithic, filesystem), Loki. PVC, resources	3개 pod Ready, 각 /ready 200	M0-02
F-M1-02	collector-grafana	Collector 파이프라인 3개, grafana.lgtm.local, datasource 3개 + correlation provisioning	Grafana API로 datasource 3개 health OK. exemplar / tracesToLogs(span 필터) / derived field 설정 존재	M1-01
F-M1-03	backend-skeleton	Go backend /healthz, stub /api/execute (가짜 compile/execute/ipfs.upload span), slog → otelslog + JSON stdout	응답 trace_id로 Tempo에서 4개 span 조회. Loki에서 같은 trace_id + span_id 로그 조회	M1-02
F-M1-04	metrics-exemplar	duration histogram + exemplar, counter, make verify-m1	Mimir exemplar API가 방금 trace_id 반환. Mimir에서 실제 metric 이름 조회 후 PRD 쿼리와 일치 확인. reader 5s. Grafana에서 metric → trace → logs 클릭 이동 (수동 1회, 스크린샷)	M1-03
F-M2-01	compiler-service	compiler 이미지 (rustc + wasm32-wasip1), POST /compile, 요청별 temp dir 삭제, 15s timeout, traceparent 수신 span	예제 코드 → 응답 앞 4바이트 \0asm. Tempo에서 backend → rustc span이 한 trace로 연결	M1-03
F-M2-02	compiler-hardening	securityContext, SA token 미마운트, env -i, read-only rootfs, NetworkPolicy (Collector만 허용), 1GB / CPU 2	scripts/verify/m2-02.sh: 7.3 케이스가 정의대로 통과. compiler pod에서 외부 curl 실패	M2-01
F-M2-03	wasmtime-exec	stub를 실제 compile + Wasmtime 실행으로 교체. capability 0, 10MB, 5s epoch, 10kB output	7.2 케이스 중 동시성 제외 5개 통과 (status 값으로 확인)	M2-01
F-M2-04	limits-queue	64KB → 413, 처리 2 + 대기 8 → 429, lgtm_queue_rejected_total	동시 20개 요청 → 429 정확히 10개. 65KB body → 413	M2-03
F-M3-01	kubo-node	Kubo + PVC, gateway → ipfs.lgtm.local, API 비노출, swarm 4001, announce 설정	add 후 gateway로 200 (subdomain redirect 301 없이 path 방식). ipfs.lgtm.local/api/v0/id 실패. swarm peers > 0	M0-02
F-M3-02	ipfs-upload	main.rs / output.txt / status.json 디렉토리 업로드 + pin, 10s timeout, ipfs.cid attribute, 실패 케이스 포함	/api/execute → cid → 세 파일 gateway 조회. compile error도 cid 있음	M2-03, M3-01
F-M3-03	external-gateway	make ipfs-check: public gateway + delegated routing 조회, NAT 진단 출력	성공/실패와 원인 출력. 결과를 README용 메모로 HANDOFF에 기록	M3-02
F-M4-01	frontend-shell	Vite + React + Monaco (로컬 번들), 예제 코드, Run, spinner, nginx 이미지, /api 라우팅	lgtm.local → Run → 실제 응답 표시. 외부 도메인 요청 0개	M0-02 (mock), M2-03

F-M4-02	result-display	FE-05~FE-10: 상태별 에러, stdout/stderr, truncated, IPFS 링크 2개 (로컬 + ipfs.io), 공개 문구, trace_id, 413/429	vitest 컴포넌트 테스트 (상태별 mock) 통과 + S1~S3 수동 체크리스트	M4-01, M3-02
F-M5-01	instrumentation	TR-01~06 attribute / error event, 지표 전체 (8.2), bucket 경계, span name 고정	verify 스크립트가 Tempo API로 attribute 존재, 실패 시 ERROR status 확인	M3-02
F-M5-02	dashboard-metrics	Dashboard 1 JSON + provisioning	Grafana /api/search에 대시보드 존재. 세 패널 쿼리 non-empty. increase 기반이라 backend 재시작 후에도 값 유지. refresh 5s	M5-01
F-M5-03	dashboard-traces	Dashboard 2, Tempo local-blocks (TraceQL metrics), 모든 duration 패널 exemplar	4개 패널 데이터 표시. Slowest는 duration 내림차순. S4 시나리오 수동 클릭 1회 성공	M5-01
F-M6-01	readme	subject VI + PRD 11장, AI 사용 내역 (HANDOFF에서 취합), 아키텍처 다이어그램	첫 줄 이탤릭 정확, 필수 섹션 3개 존재 (grep)	M5-03
F-M6-02	fresh-vm	make verify 통합, wait 로직, clean / fclean / re, 클린 스냅샷 재현	새 VM 스냅샷에서 make && make verify 한 번에 통과	전부

> **Critical path**: M0 → M1 전체 → M2-01 → M2-03 → M3-02 → M5. F-M1-04에서 exemplar가 안 뜨면 뒤가 전부 밀리므로 첫날 목표는 F-M1-04까지. worktree로 세션 두 개를 병렬로 돌린다면 F-M3-01과 F-M4-01을 M1과 병행 가능.